GUÍA DEL ALUMNO

El frontend no murió, subió de nivel.

Cualquiera puede generar un componente con un prompt. Pocos saben si ese componente está bien hecho. **Aquí empiezas a ser uno de ellos.**

Fundamentos de Vue 3
con TypeScript, Vite y Composition API

Oscar Alvarado
Frontend Tech Lead

Universidad San Martín de Porres
Lima, Perú

ANTES DE EMPEZAR

Qué vas a encontrar en esta guía

Esta guía resume la **Sesión 1** para que puedas repasar a tu ritmo. Tiene los conceptos, el código que vimos en clase, los tres ejercicios con su solución y el checklist para revisar cualquier componente, lo haya escrito una persona o una IA.

0 **Apertura**
El reto del componente de la IA

1 **Arranque**
create-vue · Vite · SFC

2 **Template**
reactividad · estilos scoped · Tailwind

3 **Directivas y eventos**
v-bind · v-if · v-for · v-model · :key · @eventos

4 **Componentes**
props · emits · ciclo de vida · composables

5 **Code review**
los 8 problemas · checklist · reto

Habilidades de la sesión

- ✓ Crear un proyecto con create-vue y Vite
- ✓ Estado reactivo con ref y computed
- ✓ Estilos scoped y utilidades tipo Tailwind
- ✓ Composables con Composition API
- ✓ Leer y escribir Single File Components
- ✓ Directivas y eventos con modificadores
- ✓ Props, emits y ciclo de vida
- ✓ Ojo de revisor: detectar errores comunes

Lo que necesitas instalado

- **Node.js 22.18 o superior** (comprueba con `node -v`).
- **VS Code** con la extensión **Vue (Official)**.
- Opcional: la extensión **Vue DevTools** del navegador.

NIVEL 0 · APERTURA 8 MIN

El reto: un componente que "funciona"

En clase le pedimos a una IA **"haz un buscador de alumnos en Vue"**. El resultado compila y se ve en pantalla. La pregunta era: **¿cuántos problemas tiene?**

BuscadorAlumnos.vue · generado por IA

```
<script setup lang="ts">
import { ref, onMounted } from 'vue'

const props = defineProps(['alumnos'])
let filtro = ''
const hora = ref('')

onMounted(() => {
  setInterval(() => {
    hora.value = new Date().toLocaleTimeString()
  }, 1000)
})

function eliminar(i) {
  props.alumnos.splice(i, 1)
}
</script>

<template>
  <input v-model="filtro" placeholder="Buscar alumno">
  <p>{{ hora }}</p>
  <ul>
    <li v-for="(a, i) in alumnos" v-if="a.nombre.includes(filtro)" :key="i">
      {{ a.nombre }}
      <div @click="eliminar(i)" style="color: red">Eliminar</div>
    </li>
  </ul>
</template>
```

Antes de ver la respuesta en el Nivel 5, intenta encontrarlos tú. Anótalos al margen.

QUÉ CAMBIÓ EN EL FRONTEND

La IA te da código que compila, un resultado que "se ve bien" y velocidad.

Tú aportas criterio para saber si está bien hecho, fundamentos (reactividad, componentes, ciclo de vida) y responsabilidad sobre lo que llega a producción.

El valor ya no está solo en escribir código, sino en **decidir si el código sirve**. Para eso hacen falta fundamentos, y eso es lo que construimos en esta sesión.

NIVEL 1 · ARRANQUE 15 MIN

create-vue, Vite y Single File Components

Crear el proyecto

`create-vue` es el generador oficial de Vue. Te hace preguntas y arma un proyecto sobre Vite.

terminal

```
npm create vue@latest mi-primer-vue
# Elige: TypeScript, Linter y Prettier
cd mi-primer-vue
npm install
npm run dev       # http://localhost:5173
```

OPCIÓN	¿HOY?	POR QUÉ
TypeScript	Sí	Los errores aparecen mientras escribes, no en producción.
Linter + Prettier	Sí	Todo el equipo escribe con el mismo estilo y se detectan errores comunes.
Router, Pinia, Vitest	Luego	Los veremos en próximas sesiones.

¿Por qué Vite?

Un bundler tradicional procesa **todos** los módulos antes de levantar el servidor. Vite levanta el servidor al instante y el navegador pide los módulos como **ES Modules nativos**, solo cuando los necesita.



- **En desarrollo:** arranque inmediato y **HMR** (Hot Module Replacement): guardas y el cambio aparece sin recargar.
- **En producción:** `npm run build` sí empaqueta y optimiza todo en `dist/`.

Anatomía del proyecto

ARCHIVO	PARA QUÉ SIRVE
`index.html`	Puerta de entrada. Carga `/src/main.ts` como módulo nativo.
`src/main.ts`	Crea la app con `createApp(App)` y la monta en `#app`.
`src/App.vue`	Componente raíz. Todo lo demás cuelga de aquí.
`src/components/`	Tus componentes. Un archivo `.vue` por componente, en PascalCase.
`src/assets/`	Imágenes y CSS que pasan por Vite (se optimizan). `public/` se copia tal cual.
`vite.config.ts`	Plugins y el alias `@` que apunta a `src/`.
`tsconfig*.json`	Reglas de TypeScript para la app y para la configuración.
`package.json`	Dependencias y scripts: `dev`, `build`, `type-check`, `lint`.
`env.d.ts`	Tipos del cliente de Vite, como `import.meta.env`.

Anatomía de un Single File Component (SFC)

Un archivo `.vue` junta lógica, vista y estilos. El navegador nunca ve un `.vue`: el compilador lo convierte en JavaScript y CSS.

Saludo.vue

```vue
<script setup lang="ts">
import { ref } from 'vue'

const nombre = ref<string>('Vue')
</script>

<template>
  <h1 class="titulo">Hola, {{ nombre }}</h1>
</template>

<style scoped>
.titulo { color: #16825a; }
</style>
```

RECUERDA

`<script setup>` es la forma recomendada: menos código repetitivo y mejor inferencia de tipos. Todo lo que declaras ahí está disponible en el template sin `return`.

El navegador recibe tres cosas distintas a partir de un solo archivo:



NIVEL 2 · TEMPLATE, REACTIVIDAD Y ESTILOS 8 MIN

Reactividad y estilos

Reactividad con ref()

`ref()` crea estado reactivo: cuando cambia, Vue vuelve a renderizar solo lo que depende de él. Una variable normal (`let`) cambia en JavaScript, pero **nadie le avisa a la vista**.



Correcto

```
const count = ref(0)
const doble = computed(() => count.value *
   2)
// en script: count.value · en template: {{
   count }}
```

Error clásico

```
let plano = 0
function inc() {
  plano++ // cambia en JS...
}
// ...pero la pantalla sigue en 0
```

REGLA

En `<script>` usas `count.value`. En el `<template>` escribes `{{ count }}`: Vue lo desenvuelve solo. Para valores derivados usa `computed`.

Estilos scoped

Con `<style scoped>`, Vue agrega un atributo único a tu HTML y a tus selectores. Así tus estilos no se escapan a otros componentes.

Lo que escribes

```
<template>
  <h2 class="titulo">Mis cursos</h2>
</template>

<style scoped>
.titulo { color: #16825a; }
</style>
```

Lo que genera Vue

```
<!-- HTML -->
<h2 class="titulo" data-v-7ba5bd90>Mis
   cursos</h2>

/* CSS */
.titulo[data-v-7ba5bd90] { color: #16825a; }
```

Sin `scoped`, si dos componentes usan `.titulo`, la última regla que carga gana en toda la aplicación.

REGLA PRÁCTICA

Usa `scoped` por defecto. Deja en `src/assets/main.css` solo lo que es de toda la app: fuentes, colores y reset.

Utilidades tipo Tailwind CSS

Con Tailwind

```
<button
  class="inline-flex items-center gap-2
   rounded-lg
         bg-emerald-700 px-4 py-2.5 font-
   bold text-white
         shadow-sm hover:bg-emerald-800"
>
  Inscribirme
</button>
```

Con CSS scoped

```
<template>
  <button class="cta">Inscribirme</button>
</template>

<style scoped>
.cta {
  display: inline-flex; align-items: center;
   gap: .5rem;
  border-radius: .5rem; padding: .625rem
   1rem;
  background: #047857; color: white; font-
   weight: 700;
}
.cta:hover { background: #065f46; }
</style>
```

	UTILIDADES (TAILWIND)	CSS SCOPED
Velocidad	Muy rápida para maquetar	Más lenta al inicio
Consistencia	Escala de espacios y colores ya definida	Depende de tus variables
Lectura	Templates con muchas clases	Template limpio
Ideal para	Layout, espaciado, tipografía	Estilos propios del componente, animaciones

No son rivales: en un mismo proyecto puedes combinar ambos. Para instalar Tailwind CSS v4 en un proyecto con Vite:

Instalación

```
# Tailwind CSS v4 con Vite
npm install tailwindcss @tailwindcss/vite

# vite.config.ts → plugins: [vue(), tailwindcss()]
# src/assets/main.css
@import "tailwindcss";
```

NIVEL 3 · DIRECTIVAS Y EVENTOS 17 MIN

Directivas y eventos

Las directivas son instrucciones que le das al template: mostrar u ocultar, repetir, enlazar atributos y escuchar eventos.

Directivas

DIRECTIVA	QUÉ HACE	EJEMPLO
`v-bind (:)`	Conecta estado con un atributo HTML	`:disabled="ancho > 200"`
`:class`	Aplica clases según el estado	`:class="{ activo: seleccionado }"`
`v-if / v-else`	Crea o destruye el elemento	`v-if="visible"`
`v-show`	Solo lo oculta con `display: none`	`v-show="visible"`
`v-for`	Repite un elemento por cada ítem	`v-for="c in cursos" :key="c.id"`
`v-on (@)`	Escucha eventos	`@click="guardar"`
`v-model`	Enlace en dos sentidos con inputs	`v-model="form.nombre"`

v-bind

```
<script setup lang="ts">
const ancho = ref(120)
const tono = ref<'verde' | 'pizarra' |
   'ambar'>('verde')
</script>

<template>
  <!-- :atributo="expresión" conecta estado
   con HTML -->
  <div :class="tono" :style="{ width: ancho
   + 'px' }">
    :style + :class
  </div>
  <button :disabled="ancho >
   200">Botón</button>
</template>
```

v-if vs v-show

```
<div v-if="visible">Hola</div>   <!-- crea /
   destruye -->
<div v-show="visible">Hola</div> <!-- solo
   oculta con CSS -->
```

v-if cuando algo cambia poco o es costoso de renderizar: el nodo no existe en el DOM. **v-show** cuando se alterna muy seguido (tabs, menús): el nodo existe pero oculto.

v-model

```
<script setup lang="ts">
const form = reactive({
  nombre: 'Ana',
  nivel: 'junior',
  acepta: true,
})
</script>

<template>
  <input v-model="form.nombre" />
  <select v-model="form.nivel">...</select>
  <input type="checkbox" v-
   model="form.acepta" />
</template>
```

v-for

```vue
<script setup lang="ts">
interface Curso { id: number; nombre: string }
const cursos = ref<Curso[]>([...])
</script>

<template>
  <ol>
    <li v-for="(curso, i) in cursos" :key="curso.id">
      {{ i }} · {{ curso.nombre }}
    </li>
  </ol>
</template>
```

:class con objeto: la clase se aplica si el valor es true

```vue
<button :class="{ activo: seleccionado, error: tieneError }">
  Guardar
</button>
```

El bug del :key

El `:key` le dice a Vue **quién es quién**. Si usas el índice, al eliminar el primer elemento Vue reutiliza el elemento equivocado: los textos se actualizan, pero el estado que Vue no controla (lo escrito en un input, el foco, una animación) se queda en la posición.

`:key="index"` · después de eliminar a Ana

Bruno	input: "Nota de Ana"
Carla	input: "Nota de Bruno"

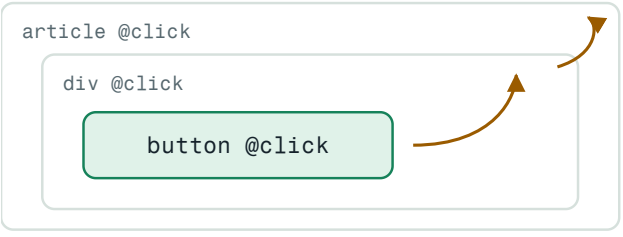
`:key="alumno.id"` · después de eliminar a Ana

Bruno	input: "Nota de Bruno"
Carla	input: "Nota de Carla"

REGLA

El `:key` debe ser **estable y único**: un `id`, nunca la posición.

Eventos y modificadores



Sin .stop: 1. button → 2. div → 3. article

Con @click.stop: solo button

El evento "burbujea" hacia los padres.
.stop llama a event.stopPropagation() por ti.

MODIFICADOR	EQUIVALE A	USO TÍPICO
`@click.stop`	`event.stopPropagation()`	Botón dentro de una tarjeta clicable
`@submit.prevent`	`event.preventDefault()`	Formularios: evita que la página se recargue
`@keyup.enter`	Filtrar por la tecla Enter	Enviar un mensaje o agregar una tarea
`@keyup.esc`	Filtrar por la tecla Escape	Limpiar o cerrar

Teclado

```
<input
  v-model="texto"
  @keyup.enter="enviar"
  @keyup.esc="texto = ''"
/>
```

Formulario

```
<!-- sin .prevent el navegador recarga la
   página -->
<form @submit.prevent="guardar">
  <input v-model="correo" type="email" />
  <button>Guardar</button>
</form>
```

NIVEL 4 · COMPONENTES 22 MIN

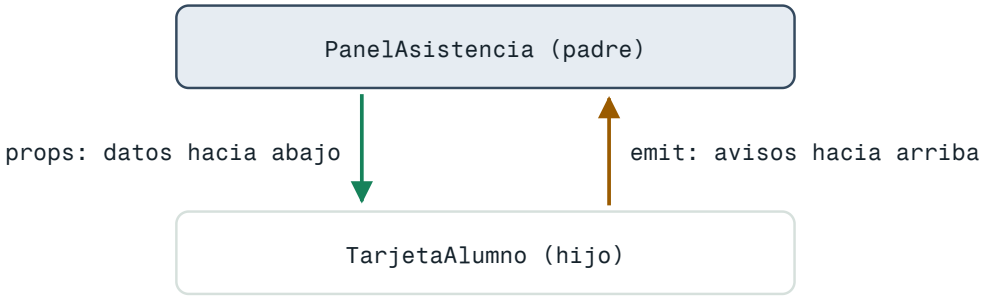
Componentes, ciclo de vida y Composition API

Pensar en componentes

Una interfaz es un árbol de piezas reutilizables. Por ejemplo, un campus virtual se puede dividir en `App` → `AppHeader`, `MenuLateral`, `TarjetaSesion` (× 3) y `BotonBase`.

- **Si se repite**, es un componente.
- **Si tiene una sola responsabilidad clara**, es un componente.
- **Nombres en PascalCase y de dos palabras**: `TarjetaSesion`, no `Tarjeta`. Así evitas choques con etiquetas HTML presentes y futuras (guía de estilo oficial de Vue).

Props y emits



Los datos **bajan** por props. Los avisos **suben** por eventos. El hijo nunca modifica una prop: le pide el cambio al padre emitiendo un evento.

TarjetaAlumno.vue (hijo)

```vue
<script setup lang="ts">
interface Alumno {
  id: number
  nombre: string
  presente: boolean
}

defineProps<{ alumno: Alumno }>()

const emit = defineEmits<{
  marcar: [id: number]
}>()
</script>

<template>
  <article>
    <strong>{{ alumno.nombre }}</strong>
    <button @click="emit('marcar',
   alumno.id)">
      Marcar asistencia
    </button>
  </article>
</template>
```

PanelAsistencia.vue (padre, template)

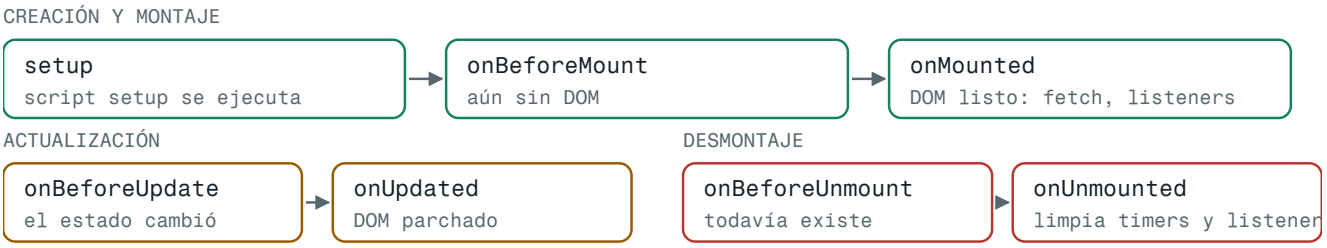
```vue
<TarjetaAlumno
  v-for="a in alumnos" :key="a.id"
  :alumno="a"
  @marcar="marcar"
/>
```

CON TYPESCRIPT

`defineProps<{ alumno: Alumno }>()` y `defineEmits<{ marcar: [id: number] }>()` hacen que el editor te avise si pasas un dato incorrecto o emites un evento que no existe.

Ciclo de vida

Todo componente nace, cambia y muere. Los hooks te permiten ejecutar código en cada momento.



Lo que abres al montar, lo cierras al desmontar

```
let timer: number | undefined
onMounted(() => { timer = window.setInterval(tick, 1000) })
onUnmounted(() => clearInterval(timer)) // si no, fuga de memoria
```

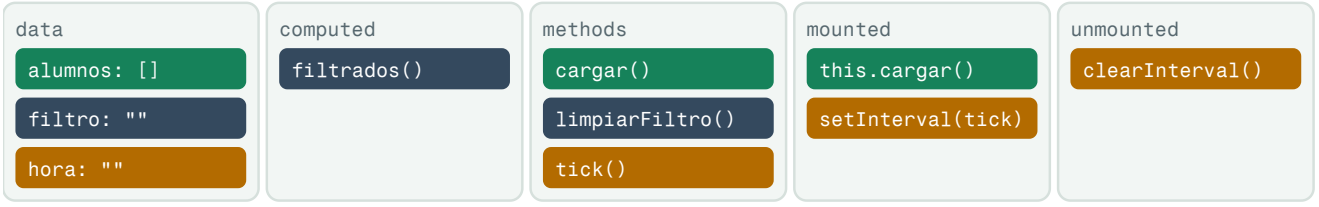
REGLA

El par `onMounted` / `onUnmounted` es el más importante. Si registras un `setInterval` o un `addEventListener` y no lo limpias, sigue vivo después de que el componente desaparece: es una fuga de memoria.

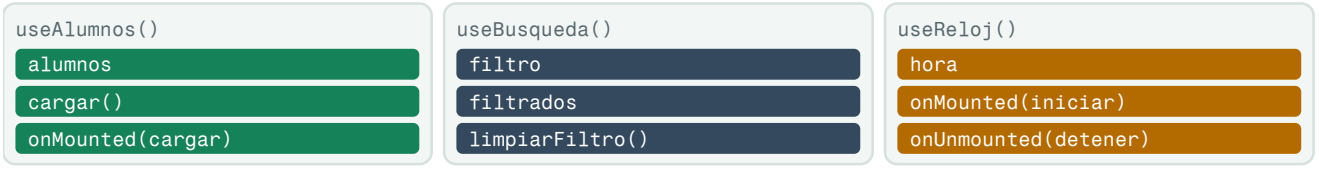
Options API vs Composition API

Con Options API, una misma funcionalidad queda repartida en `data`, `computed`, `methods`, `mounted`... Con Composition API, el código se organiza por **lo que hace**, y cada grupo puede salir a su propio archivo.

Options API: agrupado por tipo



Composition API: agrupado por funcionalidad → composables



Options API sigue siendo válida en Vue 3, pero en proyectos grandes con TypeScript se prefiere Composition API.

Composables

Un composable es una **función que reutiliza lógica con estado**.

src/composables/usePuntero.ts

```ts
import { ref, onMounted, onUnmounted, type Ref } from 'vue'

export function usePuntero(zona: Ref<HTMLElement | null>) {
  const x = ref(0)
  const y = ref(0)

  function mover(e: PointerEvent) {
    const r = zona.value!.getBoundingClientRect()
    x.value = Math.round(e.clientX - r.left)
    y.value = Math.round(e.clientY - r.top)
  }

  onMounted(() => zona.value?.addEventListener('pointermove', mover))
  onUnmounted(() => zona.value?.removeEventListener('pointermove', mover))

  return { x, y }
}

// En cualquier componente:
const zona = ref<HTMLElement | null>(null)
const { x, y } = usePuntero(zona)
```

- Su nombre empieza con `use` y se llama dentro de `<script setup>`.
- Retorna refs, así el componente mantiene la reactividad al desestructurar.
- Cada llamada crea su propio estado: `useContador(0)` y `useContador(10)` son independientes.
- Puede usar hooks del ciclo de vida, como `usePuntero`, que registra y limpia su listener.

PRÁCTICA

Ejercicios de la sesión

Los tres ejercicios construyen una misma app. Hazlos en orden y **escribe tú el código**: la solución está al final de cada uno para comparar, no para copiar.

Ejercicio 1 · Tu primer componente

5 min · Nivel 1

1. Crea el proyecto: `npm create vue@latest mi-primer-vue` con TypeScript, Linter y Prettier.
2. `cd mi-primer-vue`, `npm install` y `npm run dev`.
3. Crea `src/components/TarjetaBienvenida.vue` con un `ref` para tu nombre y un estilo `scoped`.
4. Impórtalo en `App.vue` y úsalo como `<TarjetaBienvenida />`.
+ Bonus: un botón que cuente "Me gusta" con `@click`.

Meta: ver tu nombre en `localhost:5173` y cambiarlo en el código sin recargar (HMR).

Solución

src/components/TarjetaBienvenida.vue

```vue
<script setup lang="ts">
import { ref } from 'vue'

const nombre = ref('Oscar')
const likes = ref(0)
</script>

<template>
  <article class="tarjeta">
    <h2>Hola, {{ nombre }}</h2>
    <p>Bienvenido a Vue 3 con TypeScript.
   </p>
    <button type="button" @click="likes++">
      Me gusta ({{ likes }})
    </button>
  </article>
</template>

<style scoped>
.tarjeta {
  padding: 1.5rem;
  border: 1px solid #d2ddd8;
  border-radius: 12px;
}
h2 { color: #16825a; }
</style>
```

src/App.vue

```vue
<script setup lang="ts">
import TarjetaBienvenida from
   './components/TarjetaBienvenida.vue'
</script>

<template>
  <main>
    <TarjetaBienvenida />
  </main>
</template>
```

Ejercicio 2 · Lista de pendientes del curso

6 min · Nivel 3

1. Crea `ListaPendientes.vue` con una interfaz `Tarea { id, texto, hecha }`.
2. Un `input` con `v-model` que agregue con `@keyup.enter`.
3. Muestra las tareas con `v-for` y `:key="t.id"`.
4. Un checkbox por tarea y `:class` para tacharla cuando esté hecha.
5. Mensaje con `v-if` cuando la lista esté vacía.

+ Bonus: contador de pendientes con `computed`.

Solución

src/components/ListaPendientes.vue

```vue
<script setup lang="ts">
import { ref, computed } from 'vue'

interface Tarea { id: number; texto: string; hecha: boolean }

const nueva = ref('')
const tareas = ref<Tarea[]>([
  { id: 1, texto: 'Instalar Node', hecha: true },
  { id: 2, texto: 'Crear proyecto con create-vue', hecha: false },
])

const pendientes = computed(
  () => tareas.value.filter((t) => !t.hecha).length,
)

function agregar() {
  const texto = nueva.value.trim()
  if (!texto) return
  tareas.value.push({ id: Date.now(), texto, hecha: false })
  nueva.value = ''
}
</script>

<template>
  <section>
    <input v-model="nueva" @keyup.enter="agregar"
           placeholder="Nueva tarea" aria-label="Nueva tarea" />

    <p v-if="tareas.length === 0">No hay tareas. ¡A descansar!</p>
    <ul v-else>
      <li v-for="t in tareas" :key="t.id" :class="{ hecha: t.hecha }">
        <label>
          <input type="checkbox" v-model="t.hecha" />
          {{ t.texto }}
        </label>
      </li>
    </ul>

    <small>{{ pendientes }} pendientes</small>
  </section>
</template>
```

```
<style scoped>
.hecha { text-decoration: line-through; opacity: 0.6; }
</style>
```

Fíjate en que cada tarea usa un `<label>` con checkbox y no un `<li>` clicable: así funciona con teclado y lectores de pantalla.

Ejercicio 3 · Refactoriza en componentes

6 min · Nivel 4

1. Mueve la lógica a `src/composables/useTareas.ts`: `tareas`, `pendientes`, `agregar`, `alternar`, `eliminar`.
2. Crea `TareaItem.vue` con `defineProps<{ tarea: Tarea }>()`.
3. El hijo emite `alternar` y `eliminar`. No toca la prop.
4. `ListaPendientes.vue` usa `useTareas()` y renderiza `<TareaItem>`.
+ Bonus: guarda en `localStorage` y carga en `onMounted`.

Solución

src/composables/useTareas.ts

```ts
import { ref, computed } from 'vue'

export interface Tarea { id: number; texto: string; hecha: boolean }

export function useTareas() {
  const tareas = ref<Tarea[]>([])
  const pendientes = computed(() => tareas.value.filter((t) => !t.hecha).length)

  function agregar(texto: string) {
    if (!texto.trim()) return
    tareas.value.push({ id: Date.now(), texto: texto.trim(), hecha: false })
  }
  function alternar(id: number) {
    const t = tareas.value.find((t) => t.id === id)
    if (t) t.hecha = !t.hecha
  }
  function eliminar(id: number) {
    tareas.value = tareas.value.filter((t) => t.id !== id)
  }

  return { tareas, pendientes, agregar, alternar, eliminar }
}
```

src/components/TareaItem.vue

```vue
<script setup lang="ts">
import type { Tarea } from '@/composables/useTareas'

defineProps<{ tarea: Tarea }>()
const emit = defineEmits<{
  alternar: [id: number]
  eliminar: [id: number]
}>()
</script>

<template>
  <li :class="{ hecha: tarea.hecha }">
    <label>
      <input type="checkbox" :checked="tarea.hecha"
             @change="emit('alternar', tarea.id)" />
      {{ tarea.texto }}
    </label>
    <button type="button" @click="emit('eliminar', tarea.id)">
      Eliminar
    </button>
  </li>
</template>
```

src/components/ListaPendientes.vue (template)

```vue
<TareaItem
  v-for="t in tareas" :key="t.id" :tarea="t"
  @alternar="alternar" @eliminar="eliminar"
/>
```

PUNTO CLAVE

En `TareaItem` el checkbox usa `:checked` + `@change`, no `v-model` sobre la prop. Un `v-model` sobre una prop sería modificarla desde el hijo.

NIVEL 5 · CODE REVIEW 10 MIN

Los 8 problemas del componente de la IA

Volvamos al buscador del inicio. Cada número en el código corresponde a un problema de la lista.

BuscadorAlumnos.vue · versión de la IA

```vue
<script setup lang="ts">
import { ref, onMounted } from 'vue'

const props = defineProps(['alumnos'])                    1
let filtro = ''                                           2
const hora = ref('')

onMounted(() => {
  setInterval(() => {                                     3
    hora.value = new Date().toLocaleTimeString()
  }, 1000)
})

function eliminar(i) {                                    4
  props.alumnos.splice(i, 1)                              5
}
</script>

<template>
  <input v-model="filtro" placeholder="Buscar alumno">
  <p>{{ hora }}</p>
  <ul>
    <li v-for="(a, i) in alumnos" v-if="a.nombre.includes(filtro)" :key="i">   6 7
      {{ a.nombre }}
      <div @click="eliminar(i)" style="color: red">Eliminar</div>             8
    </li>
  </ul>
</template>
```

1 Props sin tipos

Corrección: `defineProps<{ alumnos: Alumno[] }>()`. Nivel 4.

2 let filtro no es reactivo

Corrección: `const filtro = ref('')`. Nivel 2.

3 setInterval nunca se limpia

Corrección: guardar el id y llamar a `clearInterval` en `onUnmounted`. Nivel 4.

4 Parámetro sin tipo (any implícito)

Corrección: trabajar con el id del alumno: `(id: number)`. TypeScript.

5 Muta una prop con splice

Corrección: `emit('eliminar', a.id)` y que el padre decida. Nivel 4.

6 v-if y v-for en el mismo elemento

En Vue 3, `v-if` se evalúa primero y `a` todavía no existe. Corrección: una lista filtrada con `computed`. Nivel 3.

7 **:key con el índice**

Corrección: `:key="a.id"`. Nivel 3.

8 **div clicable con estilo inline**

Corrección: `<button type="button">` con una clase scoped. Niveles 2 y 3, accesibilidad.

Versión revisada

BuscadorAlumnos.vue · versión revisada

```vue
<script setup lang="ts">
import { ref, computed, onMounted, onUnmounted } from 'vue'

interface Alumno { id: number; nombre: string }

const props = defineProps<{ alumnos: Alumno[] }>()
const emit = defineEmits<{ eliminar: [id: number] }>()

const filtro = ref('')
const hora = ref('')
let timer: number | undefined

const filtrados = computed(() =>
  props.alumnos.filter((a) =>
    a.nombre.toLowerCase().includes(filtro.value.toLowerCase()),
  ),
)

onMounted(() => {
  timer = window.setInterval(() => {
    hora.value = new Date().toLocaleTimeString()
  }, 1000)
})
onUnmounted(() => clearInterval(timer))
</script>

<template>
  <input v-model="filtro" placeholder="Buscar alumno" aria-label="Buscar alumno" />
  <p>{{ hora }}</p>
  <ul>
    <li v-for="a in filtrados" :key="a.id">
      {{ a.nombre }}
      <button type="button" class="eliminar" @click="emit('eliminar', a.id)">
        Eliminar
      </button>
    </li>
  </ul>
</template>

<style scoped>
.eliminar { color: #bf3a32; }
</style>
```

NIVEL 5 · PARA LLEVAR CHECKLIST

Tu checklist de revisor

Ocho preguntas antes de aceptar cualquier componente, lo haya escrito una persona o una IA.

- [ ] ¿Lo que cambia en pantalla es `ref` o `reactive`?
  Nivel 2 · reactividad
- [ ] ¿Las props están tipadas con TypeScript?
  Nivel 4 · defineProps
- [ ] ¿Nadie modifica una prop directamente?
  Nivel 4 · emits
- [ ] ¿Cada `v-for` tiene un `:key` estable?
  Nivel 3 · :key
- [ ] ¿Filtra con `computed` en vez de `v-if` + `v-for`?
  Nivel 3 · directivas
- [ ] ¿Limpia timers y listeners al desmontarse?
  Nivel 4 · ciclo de vida
- [ ] ¿Usa elementos semánticos: botones, labels?
  Accesibilidad
- [ ] ¿Los estilos están encapsulados?
  Nivel 2 · scoped

RETO PARA LA PRÓXIMA SESIÓN

Revisa a la IA

Pídele a una IA un componente **"buscador de cursos con filtros"** en Vue 3 con TypeScript. Pásalo por este checklist.

Trae anotados los problemas que encontraste y cómo los corregiste. Los revisamos juntos al inicio de la próxima clase.

Recursos para repasar

Guía oficial de Vue	`vuejs.org/guide/introduction.html`
TypeScript con Composition API	`vuejs.org/guide/typescript/composition-api.html`
Guía de estilo de Vue	`vuejs.org/style-guide`
Guía de Vite	`vite.dev/guide`
Tailwind CSS con Vite	`tailwindcss.com/docs/installation/using-vite`
Vue Playground (practicar sin instalar nada)	`play.vuejs.org`